Q1 Count Digits in a Number

Given a number n, count how many digits it contains

Ex:

Input n = 54321

Output : 5

→ Class Demo {

Public static void main (String [] args) {

int n = 54321

int count = 0;

while (n != 0) {

int digit = n % 10;

count ++

n = n / 10;

}

System.out.print (count);

}

}

Step 1 आपण आधी while loop चालतो

लक्षात ठेवायचं आपण n % 10 = 54321 % 10 = 1

count ++ = 1

n = n / 10

① n = 54321

digit = n % 10 = 1 | count ++ = 1 | n = n/10 n = 5432

digit = n % 10 = 5432 % 10 = 2 | count ++ = 2 | n = n/10 n = 543

digit = n % 10 = 543 % 10 = 3 | count ++ = 3 | n = n/10 = 54

digit = n % 10 = 54 % 10 = 4 | count ++ = 4 | n = n/10 = 5

digit = n % 10 = 5 | count ++ = 5 | n = n/10 = 0

Q) Extract All Digits
Print all digits of a number from right to left

Ex
Input n = 5382
Output = 2
8
3
5

→ n = 5382

Step	n	n % 10	n/10
1	5382	2	538
2	538	8	53
3	53	3	5
4	5	5	0

कोशिश करो number की mod (%) 10 करने की
और number की last digit

→ Class Demo {
Public static void main(String [] args){

int n = 5382;
while (n != 0){
int digit = n % 10;
System.out.print (digit);
n = n / 10;
}
}
}

24 Reverse a Number
Reverse the digits of a number
Ex

Input 12345
Out : 54321

→ 5 x 10 = 50 + 4 = 54
54 x 10 = 540 + 3 = 543
543 x 10 = 5430 + 2 = 54321
5432 x 10 = 54320 + 1 = 54321

# Dry Run
n = 12345

int reverse = 0

① n = 12345
digit = n % 10
= 5

reverse
reverse * 10 + digit
= 0 * 10 + 45
= 45

n = n / 10
n = 1234

② n = 1234
digit = n % 10
= 4

reverse
reverse * 10 + digit
= 5 * 10 + 10
= 54

n = n / 10
= 123

③ n = 123
digit = n % 10
= 3

reverse
reverse * 10 + digit
= 54 * 10 + 3
= 543

n = n / 10
n = 12

4) n = 12
digit = n / 10
= 2

reverse =
reverse * 10 + digit
= 543 * 10 + 2
= 5432

n = n / 10
= 1

5) n = 1
digit = n / 10
= 1

reverse
reverse * 10 + digit
= 5432 * 10 + 1
= 54321

n = n / 10
= 0

code:

```
class Demo {
    Public Static void main (String [] args){

  int n = 12345 ;
  int reverse = 0 ;

  while (n != 0){
   int digit = n / 10;
   reverse = reverse * 10 + digit ;
    n = n / 10;
   }
   System out Print (digit);
  }
}
```

Q4 Palindrome Number

check if a number is palindrome if it reads the same forward & backward.

Ex:

Input: 121

Output: Palindrome

Input: 123

Output: Not Palindrome

Dry Run:

n = 121 int reverse = 0

(1) n = 121 reverse = n = n/10

digit = n % 10 reverse * 10 + digit = 12

= 1 0 * 10 + 1

= 1

(2) n = 12 reverse n = n/10

digit = n % 10 reverse * 10 + digit = 1

= 2 1 * 10 + 2

= 12

(3) n = 1 reverse n = n/10

digit = n % 10 reverse * 10 + digit = 0

= 1 12 * 10 + 1

= 121

n = reverse → Palindrome
n ! = reverse → Not Palindrome

n = 123.

int reverse = 0;

① n = 123
digit = n % 10
= 3

reverse = reverse * 10 + digit
= 0 * 10 + 3
= 3

n = n/10
= 12

② n = 12
digit = n % 10
= 2

reverse = reverse * 10 + digit
= 3 * 10 + 2
= 32

n = n/10
= 1

③ n = 1
digit = n % 10
= 1

reverse = reverse * 10 + digit
= 32 * 10 + 1
= 321

n = n/10
= 0

```
Class Demo {
    Public Static void main (String [] args){

        int n = 121;
        int reverse = 0;

        int original = n;

        while (n != 0){
            int digit = n % 10;
```

```
        reverse = reverse * 10 + digit;
        n = n/10;
      }
      if (original = reverse){
        System.out.print("Palindrome")
      }else{
        System.out.print("Not Palindrom")
    }
  }
}
```

Q5 Sum of Digits:

Find the sum of digits of a number.

Ex:

Input: 543

Output: 12

Because:

5+4+3=12

n = 543 int sum = 0

① n = 543	Sum + = digit	n = n/10
digit = n % 10	Sum = 3	= 54
= 3		
② n = 54	Sum + = digit	n = n/10
digit = n % 10	= 3+4	= 5
= 4	= 7	

b) n = 5
digit = n % 10
= 5

sum = digit
= 7+5
= 12

n = n/10
= 0

```
class Demo {
    public static void main (String [] args) {

    int n = 543;
    int sum = 10;

    while (n != 10) {
        int digit = n % 10;
        sum += digit;
            n = n/10
        }
        System.out.print ("Sum is " +sum);
    }
}
```

Q6 Armstrong Number
Check if Number is an Armstrong number
Definition:

A number is Armstrong if

Sum of (digits ^ number of digits) = numbers

Ex:
Input: 153
= $1^3 + 5^3 + 3^3$
= 153
} output Armstrong number

→ n = 153 int sum = 0

① n = 153 sum = sum + (digit * digit * digit) n = n/10
digit = n % 10 = sum + 27 = 15
= 3 = 27

② n = 15 sum = sum + digit³ n = n/10
digit = n % 10 = 27 + 125 = 1
= 5 = 152

③ n = 1 sum = sum of digit³ n = n/10
digit = n % 10 = 152 + 1 = 0
= 1 = 153

```
Class Demo{
    Public static main (String [] args){

      int n = 153
      int sum = 0; int original = n
      while (n != 0){
      int digit = n % 10
      sum = sum + (digit * digit * digit);
      n = n/10;
      }
    if (original = sum){
      System out print ("Armstrong Number")
    }else{
    System out print ("Not Armstrong Number")
```

Print All Divisors

print all divisors of a number

Ex:

Input : n=12

Out 1 2 3 4 6 12

n = 12

1×12
2×6
3×4
4×3
6×2
12×1

$\sqrt{12}, 3$

$\sqrt{12}, 2, 6, 3, 4$

$\boxed{\frac{12}{1}, \frac{12}{2}, \frac{12}{3}, \frac{12}{4}, \frac{12}{6}, \frac{12}{12}}$

```
Class Demo {
    Public static void main (String [] args) {

    int n=12;
    for (int i=1; i<=math sqrt (n); i++) {
    if (n % i == 0){
     System out print (i+" ");
     System out prin (n / i+" ");
    }
}
```

Q8 Count Number of Divisors
Find how many Divisors a number has
Ex :

Input : n = 12
Divisors : 1 2 3 4 6 12
Output : 6

int x i — 1, 2, 3

→ 1 × 12 } sop (i)
2 × 6 } sop (n/i) → 12, 6, 4
3 × 4 }

```
Class Demo {
    Public static void main (String ( Targs){

        int n = 12;
        int count = 0;

        for (int i = 1; i <= math sqrt (n); i++){
         if (n % i == 0){
            count ++;
         if (i != n/i){
            count ++;
         }
        }
    }
    System out print (count);
  }
}
```

Q9 Check Prime Numbers

Check if a number is prime or not

A number is prime if it has exactly two divisors 1 & itself

→ n = 7
count = 0

→ n % 1 → count ++
(n % n/1) → count ++

if (count == 2){
prime
}else{
not
}

```
Class Demo {
    Public static void main (String[] args){

  int n=7;
  int count=0;
 for (int i=1; i<=n; i++){
   if (n % i == 0){
      count ++;
    if ( i != n/i){
      count ++;
    }
  }
}
```

```
if (count == 2){
System.out.print ("Number is Prime")
}else{
System.out.print ("Not prime")
}
}
}
```

Q10 Print All Prime Numbers up to N
Print all Prime Numbers from 1 to n

Input : n = 20
Output 2 3 5 7 11 13 17 19

n = 20

- for loop 2 घालून सुरू करायचा आहे
- outer loop च्या आत count declare करायचा
- मग inner loop मध्ये condition check करायचे

```
if i % j == 0){
        count ++;
        }
```

आणि outer loop ला % लावून (count == 2){
Sop (i + " ")
}

म्हणजे ते प्रत्येक i च्या value साठी check करणार

```
# Code
class Demo {
    Public static void main (String [] Jargs) {

        int n=20;

    for (int i=2; i<n; i++){
        int count=0
    for (int j=1; j<i; j++){
            if (i% j==0){
                count ++;
        }
    }
    if (count==2){
        System out print (i+" ");
        }
    }
}
}
```

Q1 Find the GCD / HCF of two Numbers

Input:
a = 12
b = 8

Out put
6

12 = 1, 2, 3, 4, 6, 12 } 1, 2, 4, 8
18 = 1, 2, 3, 6, 9, 18 } = 6

→ a = 12
b = 18

① Divisor
12 = 1, 2, 3, 4, 6, 12
18 = 1, 2, 3, 6, 9, 18

② Common गादायपि
= 1, 2, 3, 6

③ Greatest गादायपि
= 6

Code

```
Class Demo {
    Public Static void main (String [] Args) {

        int a = 12 ;
        int b = 18 ;
        int gcd = 1 ;

    for (int i = 1 ; i < Math.min (a,b) ; i++) {
        if (a % i == 0 && b % i == 0) {
            gcd = i ;
        }
    }
    System.out.print (gcd) ;
    }
}
```

Q12 GCD using Euclidean Algorithm
Find GCD using the Euclidean Algorithm.

Input
a = 48
b = 18

Output 6

→ gcd (a, b) = gcd (b, a % b)

gcd (48, 18) = gcd (a % b, 18)
= (48 % 18, 18)
= (12, 18)
= (18, 12)
= (18 % 12, 12)
= (6, 12)
= (12, 6)
= (12 % 6, 6)
= (0, $\boxed{6}$)

```
Class Demo {
    Public static void main (String [] args){

  int a = 48;
  int b = 18;
  int gcd = 1;

  while (a != 0 && b != 0){
      if (a>b){
        a = a % b;
```

```
        } else {
          b = b % a;
        }
      }
      if (a == 0) {
        gcd = b;
      } else {
        gcd = a;
      }
        System.out.print(gcd);
    }
}
```

Q13 Check Coprime Numbers

Two numbers are coprime if there GCD = 1

Input: 8 15

Output: Coprime

a = 8

b = 15

gcd(8, 15) = gcd(15 % 8, 8)

= gcd(7, 8)

= gcd(8, 7)

= gcd(8 % 7, 7)

= gcd(1, 7)

= gcd(7, 1)

= gcd(7 % 1, 1)

= gcd(0, 1)

```
Class Demo {
    public static void main (String [] args) {

    int a=8      int b=15
    int gcd = 0;

      while (a != 0 && b != 0){
          if (a>b){
            a = a % b;
          }
          else {
          b = b % a;
        }
      }
    if (a == 0){
        gcd = b ;
      }else{
        gcd = a;
      }

    if (gcd == 1){
      System.out.print ("Coprime");
      }else{
        System.out.print ("Not Coprime");
    }
  }
}
```

4 Check Perfect Number.

A Number is perfect if

Sum of its proper divisors = number

Input 6

Divisors 1 2 3

1 + 2 + 3 = 6

Output

Perfect Number

n/2 कोणत्याही number का perfect number है

n = 6

sum = 0.

n/2 कोणत्याही number की perfect divisor n/2 पेक्षा मोठा असला expect नाही

for loop

```
Class Demo {
    Public static void main (String [] args){

        int n=6
        int sum=0;

    for (int i = 1; i < n/2 ; i++){
        if (n % i == 0){
            sum += i;
        }
    }
    if (sum == n){
        System.out.print ("Perfect number");
    }
    else {
        System.out.print ("Not Perfect ");
    }
}
```